JTAGprobe_smp_pio_std — Architecture & Implementation

CMSIS-DAP v2 JTAG/SWD probe on the Raspberry Pi Pico (RP2040), µOS++ III SMP, PIO-accelerated JTAG

This document describes how the firmware in this repository is configured and how it works, layer by layer: the board wiring, the build, the µOS++ III SMP kernel and its RP2040 port, the USB transport and the packet path between the two cores, the CMSIS-DAP command layer, and the PIO state machine that clocks JTAG. Every statement refers to the source as it is at commit `ad73e50`. Performance figures are the ones measured by the project and recorded in `README.md` and `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md`; they were not re-measured for this document.

Contents

1. Overview
2. System architecture
3. Hardware configuration (pinout and pin ownership)
4. Build configuration
5. The µOS++ III SMP layer
6. USB transport and the inter-core packet path
7. The CMSIS-DAP layer
8. The PIO JTAG engine
9. The SWD path
10. The USB CDC-ACM UART bridge
11. Performance summary
12. Using the probe
13. Observations and known caveats
14. Source map

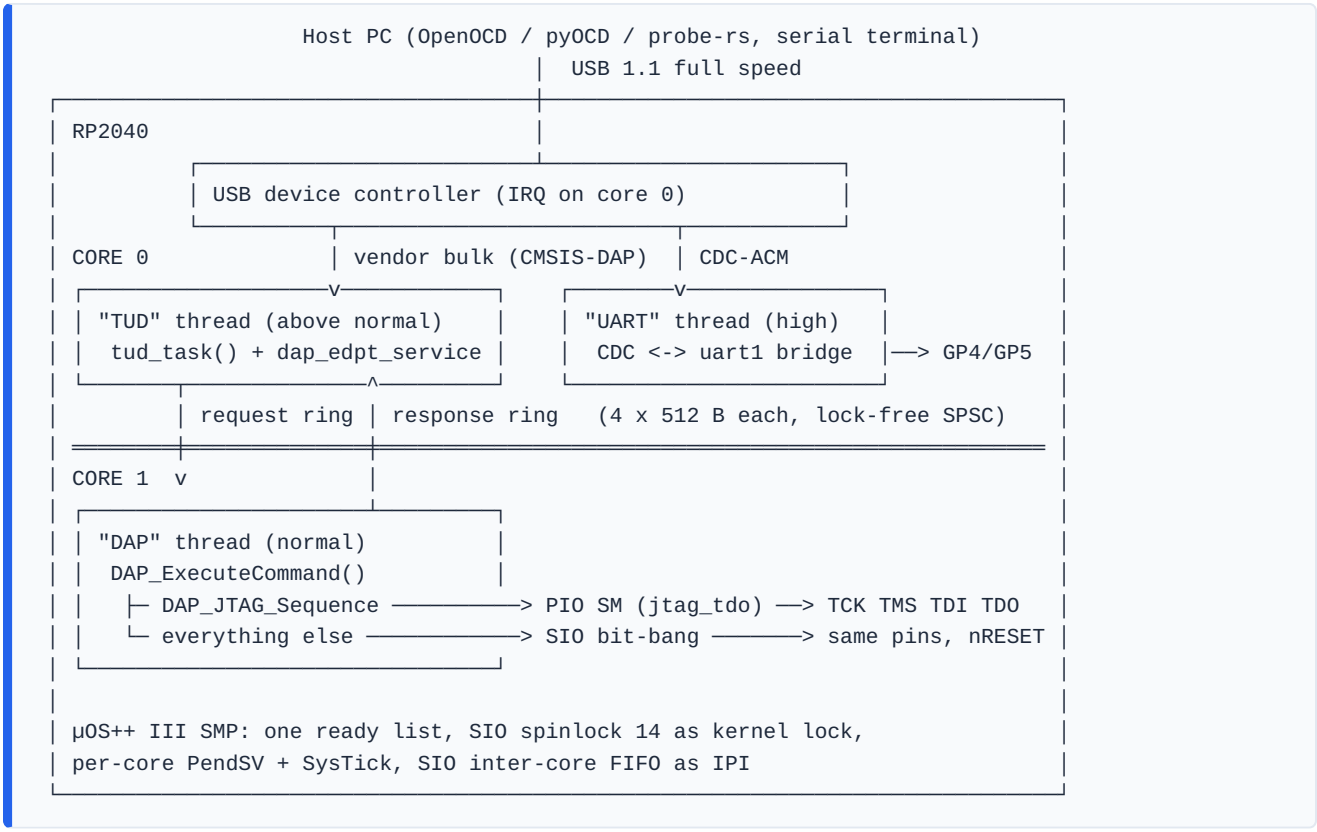
1. Overview

`JTAGprobe_smp_pio_std` turns a Raspberry Pi Pico into a CMSIS-DAP v2 debug probe with these design choices:

Aspect	Choice	Where
Operating system	µOS++ III, SMP branch, running **one scheduler across both Cortex-M0+ cores**	`micro-os-plus/`
Work split	Core 0: USB (tinyusb) and the UART bridge. Core 1: CMSIS-DAP command execution only	`src/main.c`
JTAG clocking	`DAP_JTAG_Sequence` is executed by an **RP2040 PIO state machine**; everything else bit-bangs through SIO	`src/jtag.pio`, `src/pio_jtag.c`
Packet size	512-byte DAP packets carried in 64-byte full-speed bulk packets	`include/DAP_config.h`, `src/tusb_edpt_handler.c`
Wiring	"std" layout: everything on Pico header pins 1–10, one 10-way ribbon	`include/DAP_config.h`, `include/board_pico_config.h`

The probe enumerates as USB `2e8a:000c` ("JTAG probe on Pico (CMSIS-DAP)"), the same IDs as Raspberry Pi's debugprobe, so stock OpenOCD, pyOCD and probe-rs configurations work unchanged.

2. System architecture



Layering, bottom-up:

1. **Pico SDK** boots the chip (crt0, clocks, runtime init, `copy_to_ram`) and provides the hardware drivers (GPIO, PIO, UART, timer, multicore).
2. **µOS++ III SMP** kernel (unmodified) plus the **RP2040 port** (`micro-os-plus-iii-rp2040`) supply threads, mutexes, semaphores and the dual-core scheduler.
3. **tinyusb** (built with `OPT_OS_NONE`) supplies the USB device stack; the probe adds its own class driver for the CMSIS-DAP bulk interface.

4. **ARM CMSIS-DAP** reference code (`DAP.c` , `JTAG_DP.c` , `SW_DP.c` , `SWO.c`) adapted through `DAP_config.h` .
5. **PIO JTAG engine** (`jtag.pio` , `pio_jtag.c`) hooked into `DAP_JTAG_Sequence` .

3. Hardware configuration

3.1 Pinout ("std" layout)

All signals sit on one row of the Pico header, pins 1–10, so a single 10-way ribbon cable reaches the target.

Pico pin	GPIO	Signal	Direction / driver	Notes
1	GP0	nTRST	Output, SIO	Driven high when JTAG starts; see § 13.2
2	GP1	nRESET	Open-drain emulation, SIO	Released = input with pull-up; asserted = driven low
3	—	GND		
4	GP2	SWCLK / TCK	SWD: SIO. JTAG: PIO side-set	
5	GP3	SWDIO / TMS	SWD: SIO (bidirectional). JTAG: PIO `SET`	
6	GP4	UART TX → target RX	`uart1`	
7	GP5	UART RX ← target TX	`uart1`	
8	—	GND		
9	GP6	TDI	JTAG: PIO `OUT` ; high-Z in SWD	
10	GP7	TDO	Input, pull-up, PIO `IN` (sync bypassed)	
—	GP25	On-board LED	`PROBE_USB_CONNECTED_LED` , SIO	On; blinks at 10 Hz while TCK/SWCLK is in use (§ 10.1)

The pin numbers are defined in three places, which must agree:

File	Macros	Used by
`include/DAP_config.h`	`PROBE_PIN_SWCLK_TCK` (2), `PROBE_PIN_SWDIO_TMS` (3), `PROBE_PIN_TDI` (6), `PROBE_PIN_TDO` (7), `PROBE_PIN_nTRST` (0), `PROBE_PIN_nRESET` (1)	SIO pin helpers, PIO configuration (`g_pio_jtag`)
`include/DAP.h`	`DAP_SWJ_SWCLK_TCK` (2), `DAP_SWJ_SWDIO_TMS` (3), `DAP_SWJ_TDI` (6), `DAP_SWJ_TDO` (7), `DAP_SWJ_nTRST` (0), `DAP_SWJ_nRESET` (1)	`DAP_SWJ_Pins` command (see § 13.1)
`include/board_pico_config.h`	`PROBE_UART_TX` (4), `PROBE_UART_RX` (5), `PROBE_UART_INTERFACE` (`uart1`), `PROBE_UART_BAUDRATE` (115200)	UART bridge

GP4/GP5 are UART1's function pins on the RP2040, which is why the bridge uses `uart1` .

3.2 Who owns each pin, per mode

The RP2040 GPIO function mux selects one owner per pin (SIO, PIO0/1, UART…). The firmware moves TCK, TMS and TDI between SIO and the PIO at run time:

State	TCK (GP2)	TMS (GP3)	TDI (GP6)	TDO (GP7)	nTRST (GP0)	nRESET (GP1)
After `DAP_SETUP()` (boot)	SIO out, high	SIO out, high	SIO out, high	SIO in	SIO in (reset state)	SIO in, latch high
`DAP_Connect(JTAG)` → `PORT_JTAG_SETUP` → `pio_jtag_init()`	PIO side-set, high	PIO set, high	PIO out, high	input, pull-up, sync bypass	SIO out, high	SIO in, pull-up, latch low
During a `DAP_JTAG_Sequence`	PIO	PIO	PIO	PIO reads	SIO	SIO
Other JTAG command (`DAP_Transfer`, IR scan, IDCODE, ABORT, `SWJ_Sequence`, `SWJ_Pins`)	SIO (handed over at current level)	SIO	SIO	SIO reads	SIO	SIO
`DAP_Connect(SWD)` → `PORT_SWD_SETUP`	SIO out (PIO released)	SIO, bidirectional	high-Z	high-Z	high-Z	SIO out, high
`DAP_Disconnect` → `PORT_OFF`	high-Z	high-Z	high-Z	high-Z	high-Z	high-Z

The hand-over (§ 8.6) is glitch-free: the SIO output latches are loaded with the pins' current levels before the function mux is switched.

3.3 Electrical notes

- All I/O is 3.3 V; the target must be 3.3 V or level-shifted.
- nRESET is emulated open-drain: `PIN_nRESET_OUT(1)` makes it an input (the internal pull-up or the target's pull-up holds it high), `PIN_nRESET_OUT(0)` drives it low. It is never driven high.
- TDO has the internal pull-up enabled, so an unconnected TDO reads as all ones (the classic "IDCODE 0xFFFFFFFF" symptom of a broken chain).
- Connect at least one GND (pins 3 and 8 are both ground).

4. Build configuration

4.1 Top-level CMake (`CMakeLists.txt`)

Setting	Value	Why
`cmake_minimum_required`	3.20	The µOS++ CMake files need it
`TINYUSB_OPT_OS`	`OPT_OS_NONE`	tinyusb uses no OS primitives; the probe serialises every tinyusb call with its own µOS++ mutex (`usb_mutex`)
`add_subdirectory(micro-os-plus/micro-os-plus-iii-rp2040)`		Brings in the kernel and the port as INTERFACE libraries
Sources	`DAP.c` , `JTAG_DP.c` , `SW_DP.c` , `SWO.c` , `DAP_vendor.c` , `main.c` , `usb_descriptors.c` , `tusb_edpt_handler.c` , `cdc_uart.c` , `get_serial.c` , `pio_jtag.c`	
`pico_generate_pio_header(... src/jtag.pio)`		Assembles the PIO program into `jtag.pio.h` (`jtag_tdo_program` , `jtag_tdo_program_get_default_config()`)
Includes	`CMSIS_6/CMSIS/Core/Include/` (git submodule), `include/` , `src/`	`src/` holds `cmsis-plus/os-app-config.h`
`PICO_RP2040_USB_DEVICE_ENUMERATION_FIX=1`		RP2040-E5 errata workaround
`OS_USE_OS_APP_CONFIG_H`		Makes the kernel read `os-app-config.h`
`PICO_USE_MALLOC_MUTEX=0`		The kernel's `malloc()` already runs under the scheduler lock across both cores; the SDK's wrapper must not add a Pico mutex on top
Link	`micro-os-plus::rp2040` , `pico_stdlib` , `pico_unique_id` , `tinyusb_device` , `tinyusb_board` , `hardware_pio`	
`pico_set_binary_type(... copy_to_ram)`		The whole image is copied to SRAM at boot: no XIP cache misses in the timing-critical loops

4.2 The RP2040 port's CMake (`micro-os-plus/micro-os-plus-iii-rp2040/CMakeLists.txt`)

- Locates the kernel in `../micro-os-plus-iii` (override: `-DUOS_SMP_DIR=…`) and adds it as `micro-os-plus::iii` .

- **Removes the kernel's `src/libc/stdlib/exit.c`** from the kernel's interface sources: the Pico SDK supplies `exit()`.
- Adds `src/os-core-rp2040.cpp` (context switch, tick, lock, IPI, core-1 launch).
- Defines `OS_NCPU=2` and `OS_USE_SMP_SCHEDULER=1`.
- Compiles C++ with `-fno-use-cxa-atexit` (static destructors are not registered; the firmware never exits).
- Links with `--defsym=_Heap_Begin=__end__` and `--defsym=_Heap_Limit=__HeapLimit` so the kernel's `_sbrk()` uses the heap region of the SDK linker script.
- Depends on the SDK's `cmsis_core`, `hardware_sync`, `hardware_timer`, `hardware_clocks`, `pico_multicore`.

4.3 Kernel configuration (`src/cmsis-plus/os-app-config.h`)

Macro	Value
`OS_INTEGER_SYSTICK_FREQUENCY_HZ`	1000 (1 tick = 1 ms)
`OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES`	2048
`OS_INTEGER_RTOS_IDLE_STACK_SIZE_BYTES`	1024 (core-0 idle; core-1 `idle1` has 512 B)
`OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES`	2048

4.4 Building

```
export PICO_SDK_PATH=/path/to/pico-sdk
git submodule update --init          # CMSIS_6
cmake -B build -G Ninja -DPICO_BOARD=pico
ninja -C build
# -> build/JTAGprobe_smp_pio_std.uf2 (BOOTSEL drag-and-drop), .elf (symbols)
```

The current build is 64,796 bytes of text and 40,232 bytes of BSS (`arm-none-eabi-size`), all of it resident in the RP2040's 264 KB SRAM.

5. The µOS++ III SMP layer

5.1 Why an SMP RTOS

The RP2040 has two Cortex-M0+ cores, no data caches (so no coherency management is needed) and a shared SIO block with hardware spinlocks and inter-core FIFOs. The project uses that to run **one** µOS++ scheduler over both cores, rather than two independent systems: the same mutex and semaphore objects work from either core, and a thread's core is chosen by an affinity mask.

The kernel in `micro-os-plus/micro-os-plus-iii/` is an unmodified `git archive` of µOS++ III, branch `smp`, commit `931f1380`. Everything RP2040-specific is in `micro-os-plus/micro-os-plus-iii-rp2040/`.

5.2 Boot sequence

```
Reset –> SDK crt0 (core 0): copy image to RAM, clocks, runtime init
      –> __libc_init_array()
           └─ os_rp2040_initialize_free_store()  [constructor(101)]
                gives the kernel free store __end__ .. __HeapLimit
      –> kernel main()  (micro-os-plus-iii/src/rtos/os-main.cpp)
           ├─ scheduler::initialize()           (port: disable IRQs)
           ├─ creates "main" thread running os_main(), core-0 idle thread
           └─ scheduler::start()                (port, core 0)
                ├─ MSP -> PSP, MSP reset to vector-table top
                ├─ fills the interrupt stack with the magic word
                ├─ PendSV at lowest priority, fake current thread
                ├─ drains SIO FIFO (FIFO IRQ not yet enabled)
                └─ pend PendSV, enable IRQs  –> first context switch
      –> os_main()  (src/main.c, "main" thread, core 0)
           ├─ DAP_Setup(), board_init(), led_init(), usb_serial_init()
           ├─ create usb_mutex, usb_sem, dap_sem, uart_sem
           ├─ tusb_init(), cdc_uart_init()
           ├─ create threads UART (core 0), TUD (core 0), DAP (core 1)
           ├─ os_rp2040_start_core1()
           │    ├─ creates "idle1" (512 B stack), registers it as core-1 idle
           │    ├─ multicore_reset_core1(); multicore_launch_core1_raw(
           │    │      port_smp_secondary_start, __StackOneTop, VTOR)
           │    └─ drains FIFO, enables SIO_IRQ_PROC0 (core-0 IPI)
           └─ os_thread_join(TUD)   (never returns)

Core 1: port_smp_secondary_start()
      ├─ PendSV + SysTick (1 kHz) at lowest priority
      ├─ drains FIFO, enables SIO_IRQ_PROC1 (core-1 IPI)
      ├─ fake current thread, MSP -> PSP
      └─ pend PendSV, enable IRQs –> picks the DAP thread (or idle1)
```

The IPI interrupt on core 0 is enabled only *after* the core-1 launch: `multicore_launch_core1_raw()` performs its handshake over the same SIO FIFO, and an active FIFO handler would swallow its words.

5.3 The RP2040 port

Function	Implementation	File
CPUs	`OS_NCPU=2`, `OS_USE_SMP_SCHEDULER=1`; `port_cpu_id()` reads `SIO_CPUID`	`os-core-rp2040.cpp`, `os-inlines.h`
Context switch	Naked `PendSV_Handler`, lowest priority, one per core. ARMv6-M variant: `stmia` / `ldmia` reach only r0–r7, so r8–r11 and `EXC_RETURN` are moved through r3–r7	`os-core-rp2040.cpp`
Thread frame	9 software-saved words (r4–r11, EXC_RETURN) + 8 hardware words; new threads start with `EXC_RETURN = 0xFFFFFFFD` (thread mode, PSP) and the xPSR T-bit set	`context::create()`
Kernel lock	Recursive `_smp_klock` (owner, depth) on top of **SIO hardware spinlock 14** (`PICO_SPINLOCK_ID_OS1`, the one the SDK reserves for an RTOS)	`os-inlines.h`
Critical sections	PRIMASK (`cpsid i`), taken *before* reading the CPU id so a PendSV cannot migrate the thread between the read and the per-core state write	`scheduler::locked()`
Thread claim	`switch_stacks()` runs under the kernel lock; the incoming thread's `stack_ptr` is zeroed to mark it live, so the other core's pick loop skips it	`switch_stacks()`
Tick	SysTick on both cores at 1 kHz. Core 0 calls `os_systick_handler()` (advances the clock, timeouts); core 1's SysTick only requests a reschedule, so it picks up threads woken by core 0	`SysTick_Handler`
IPI	`port_smp_ipi(cpu)` writes a token (`0x4D53`, "SM") into the SIO FIFO towards the other core (non-blocking: a full FIFO already means an IPI is pending) and executes `SEV`; the receiving core's `SIO_IRQ_PROCn` handler drains its FIFO and pends PendSV	`port_smp_ipi`, `SIO_IRQ_PROC0/1_Handler`
Idle threads	Kernel idle on core 0; `idle1` on core 1 (`WFI` + yield). The kernel keeps a thread named `idle1` on core 1	`secondary_idle_func`
High-resolution clock	`time_us_64()` (1 MHz SDK timer, shared by both cores)	port
Free store	`__end__` … `__HeapLimit`, installed by a priority-101 constructor before any C++ allocation	`os_rp2040_initialize_free_store`
Exit	SDK `exit()`; `os_run_fini_array()` is empty	

5.4 Threads and placement

Created by `create_thread()` in `src/main.c`, which sets the stack, priority and **CPU affinity mask** (`th_cpu_affinity`):

Thread	Function	Priority	Affinity	Stack	Role
`UART`	`cdc_thread()`	`os_thread_priority_high`	core 0	2 KB	CDC ↔ `uart1` bridge. Must preempt `TUD`, otherwise UART characters are lost
`TUD`	`usb_thread()`	`os_thread_priority_above_normal`	core 0	2 KB	`tud_task()`, then `dap_edpt_service()`
`DAP`	`dap_thread()`	`os_thread_priority_normal`	core 1	2 KB	Runs every CMSIS-DAP command
`main`	`os_main()`	kernel default	core 0	2 KB	Initialisation, then joins `TUD` forever
idle / `idle1`	kernel / port	idle	0 / 1	1 KB / 512 B	`WFI`

Core 1 holds only `DAP` (and `idle1`), so DAP execution is never preempted by anything but the core-1 tick. On core 0 priority preemption puts the UART bridge ahead of USB.

5.5 Synchronisation objects (`src/probe_os.h`)

Object	Type	Posted by	Waited on by	Purpose
`usb_mutex`	mutex	—	everyone calling tinyusb	tinyusb is not thread-safe and is called from both cores; every call is made with this mutex held
`usb_sem`	binary semaphore	`tud_event_hook_cb()` (USB IRQ), DAP thread when it cannot take `usb_mutex`	`TUD` (timed, 1 tick)	Wakes the USB thread
`dap_sem`	binary semaphore	OUT-complete and IN-complete callbacks	`DAP` (after 2 ms of spinning)	New request, or a response slot freed
`uart_sem`	binary semaphore	CDC line-coding / line-state / break callbacks	`UART` (timed, baud-rate-derived interval)	Wakes the bridge

Design rule: the packet path stays out of the kernel. While packets are flowing, the DAP and TUD threads spin instead of sleeping, and requests and responses cross cores through lock-free rings (§ 6.3). The first SMP build slept and woke across cores on every packet and was 4× slower; the semaphores are now only used to go idle and to come back.

6. USB transport and the inter-core packet path

6.1 USB device (`src/usb_descriptors.c` , `src/tusb_config.h`)

Field	Value
VID:PID	`0x2E8A:0x000C` (Raspberry Pi, CMSIS-DAP debug probe)
`bcdDevice`	`0x0200`
Product string	`JTAG probe on Pico (CMSIS-DAP)` (contains "CMSIS-DAP", which hosts use to detect the probe)
Serial number	From the flash unique ID (`get_serial.c` , `pico_unique_id`)
Interface 0	Vendor-specific, 2 bulk endpoints, 64-byte max packet: **CMSIS-DAP v2**. Kept at interface 0 because old Keil MDK versions look only there
Interfaces 1–2	CDC-ACM (notification + data): the UART bridge, `/dev/ttyACMx` / `COMx`
MS OS 2.0 descriptor	Served by `tud_vendor_control_xfer_cb()` (vendor request 1, `wIndex` 7) so Windows binds WinUSB automatically
CDC buffers	RX 128 B, TX 4096 B

The CMSIS-DAP interface does not use tinyusb's vendor class: the probe registers its own class driver (`_dap_edpt_driver` , via `usbd_app_driver_get_cb()`) so it controls endpoint arming directly.

6.2 512-byte DAP packets over 64-byte USB packets

`DAP_PACKET_SIZE` is 512 (CMSIS-DAP v2) and `DAP_PACKET_COUNT` 4. OpenOCD sizes its command queue to the packet size, so one round trip carries eight times more work than with 64-byte packets. The full-speed bulk endpoint is still 64 bytes, which creates two framing problems:

- **OUT (requests):** a bulk transfer ends on a short packet, but the host sends no ZLP after a request that is an exact multiple of 64 bytes. The OUT endpoint is therefore armed **one 64-byte packet at a time**, and after each full packet `dap_req_len()` parses the DAP command(s) received so far to decide whether the request is complete. It knows the length rule of every command, including the variable-length ones (`DAP_Transfer` , `DAP_TransferBlock` , `DAP_SWJ_Sequence` , `DAP_SWD_Sequence` , `DAP_JTAG_Sequence` , `DAP_JTAG_Configure`), and recurses into `DAP_QueueCommands` / `DAP_ExecuteCommands` .
- **IN (responses):** the host reads up to 512 bytes, so a response whose length is a 64-byte multiple but shorter than 512 is followed by a ZLP (`in_zlp`).

6.3 The rings

```
typedef struct {
    uint8_t  data[DAP_PACKET_COUNT][DAP_PACKET_SIZE];   // 4 x 512 B
    uint16_t resp_len[DAP_PACKET_COUNT];
    volatile uint32_t wptr;                             // free-running
    volatile uint32_t rptr;
} buffer_t;
```

Ring	Producer	Consumer
`USBRequestBuffer`	OUT callback (`TUD` , core 0)	`DAP` thread (core 1)
`USBResponseBuffer`	`DAP` thread (core 1)	IN path (`TUD` , or `DAP` while holding `usb_mutex`)

Each index has exactly one writer, so no lock is needed; a `__dmb()` orders a slot's data before the index that publishes it, and the consumer issues a `__dmb()` after observing the index. Indices run free and are reduced modulo `DAP_PACKET_COUNT`; "full" is `wptr - rptr >= 4`.

If every request slot is full, the OUT endpoint is left unarmed (`out_stalled`), so the host is NAKed; it is re-armed as soon as the DAP thread frees a slot.

6.4 Life of one DAP packet

```
Host        USB IRQ / TUD (core 0)                  DAP thread (core 1)
 | OUT 64B -> dap_edpt_xfer_cb(OUT)
 |            out_len += 64; dap_req_len() says "more"
 |            └ dap_arm_out()  (next 64 B, same slot)
 | OUT ≤64B -> request complete
 |            __dmb(); USBRequestBuffer.wptr++ ──────> (spinning on wptr)
 |            arm OUT for next slot (or out_stalled)   __dmb(); copy slot -> DAPRequestBuffer
 |            os_semaphore_post(dap_sem)               rptr++; if out_stalled: dap_kick_usb()
 |                                                     DAP_ExecuteCommand()   <-- JTAG/SWD work
 |                                                     wait for a free response slot
 |                                                     copy -> response slot; __dmb(); wptr++
 |                                                     dap_kick_usb():
 |                                  ┌──────────────── try_lock(usb_mutex) ok -> dap_edpt_service()
 |                                  |                  else post usb_sem (TUD will do it)
 | <── IN ─── usbd_edpt_xfer(IN, slot, len)
 |            dap_edpt_xfer_cb(IN): ZLP if needed,
 |            rptr++, start next IN, post dap_sem
```

Two latency optimisations are visible here:

- **Spin-then-sleep.** `dap_wait_change()` busy-waits up to `DAP_SPIN_US` = 2 ms for the ring index to move before it blocks on `dap_sem`. Core 1 has nothing else to do, so spinning costs nothing and avoids a sleep, an IPI and two context switches per packet. Likewise the `TUD` thread keeps calling `tud_task()` while `dap_edpt_active()` is true (rings not empty, or a packet moved within `USB_SPIN_US` = 2 ms).
- **The DAP thread starts its own IN transfer** when `usb_mutex` is free (`dap_kick_usb()`), instead of waking `TUD`.

`DAP_QueueCommands` packets are held back (rewritten to `DAP_ExecuteCommands`) until a non-queue packet arrives, as the CMSIS-DAP atomic-command semantics require.

6.5 Packet timing counters (vendor command `0x80`)

`tusb_edpt_handler.c` keeps per-stage timings in µs (1 MHz timer, shared by both cores), read back through `DAP_ProcessVendorCommand()` (`src/DAP_vendor.c`):

Index	Field	Meaning
0	`packets`	Requests executed
1	`queue_us`	Σ OUT received → DAP thread starts it
2	`exec_us`	Σ `DAP_ExecuteCommand()` time
3	`exec_max_us`	Longest execution
4	`slot_wait_us`	Σ DAP waiting for a free response slot
5	`inq_us`	Σ response ready → IN started
6	`in_us`	Σ IN started → IN complete
7	`in_n`	IN transfers
8	`gap_us`	Σ OUT → next OUT (gaps < 10 ms)
9	`gap_n`	Such gaps
10	`gap_idle_n`	Gaps ≥ 10 ms (host idle)
11	`out_stalls`	OUT left unarmed because the request ring was full

```
cmsis-dap cmd 0x80 0xff    ;# reset all counters
cmsis-dap cmd 0x80 2       ;# read exec_us -> "Returned data b0 b1 b2 b3" (little-endian)
```

7. The CMSIS-DAP layer

7.1 Origin and adaptation

`src/DAP.c`, `JTAG_DP.c`, `SW_DP.c`, `SWO.c` and `include/DAP.h` are ARM's CMSIS-DAP reference implementation (v2.1). All hardware access goes through `include/DAP_config.h`, which this project fills in for the RP2040:

Setting	Value	Meaning
`DAP_SWD`, `DAP_JTAG`	1, 1	Both transports
`DAP_DEFAULT_PORT`	1 (SWD)	Used by `DAP_Connect(autodetect)`
`DAP_DEFAULT_SWJ_CLOCK`	1 MHz	Until the host sends `DAP_SWJ_Clock`
`DAP_JTAG_DEV_CNT`	8	Devices on the scan chain
`DAP_PACKET_SIZE` / `COUNT`	512 / 4	(64 if built as CMSIS-DAP v1/HID)
`CPU_CLOCK`	125,000,000	Basis of the bit-bang delay calculation
`IO_PORT_WRITE_CYCLES`	1	SIO writes are single-cycle on the M0+
`SWO_UART`, `SWO_MANCHESTER`, `DAP_UART`	0	SWO and the DAP UART are not offered (the UART is the separate CDC interface)
`TIMESTAMP_CLOCK`	1 MHz	`TIMESTAMP_GET()` = `time_us_32()`
`USE_PIO_JTAG`	1	Enables the PIO engine

Pin primitives (`PIN_SWCLK_TCK_SET()` , `PIN_TDI_OUT()` , `PIN_TDO_IN()` …) are single SIO register writes/reads (`gpio_set_mask` , `gpio_clr_mask` , `gpio_get_all`), forced inline.

7.2 Command dispatch

The DAP thread calls `DAP_ExecuteCommand()` , which handles `DAP_ExecuteCommands` and otherwise calls `DAP_ProcessCommand()` , a `switch` on the command ID. Every handler returns `(request bytes consumed << 16) | response bytes written` .

How each relevant command is executed:

Command	JTAG mode	SWD mode
`DAP_Connect`	`PORT_JTAG_SETUP()` → `pio_jtag_init()`	`PORT_SWD_SETUP()` → `pio_jtag_deinit()` + SIO setup
`DAP_Disconnect`	`PORT_OFF()` : PIO released, all pins high-Z	same
`DAP_SWJ_Clock`	Sets the bit-bang delay **and** the PIO divider (`pio_jtag_set_clock()`)	Bit-bang delay
`DAP_JTAG_Sequence`	**PIO: whole command streamed by `pio_jtag_sequences()`**	—
`DAP_JTAG_Configure`	Stores IR lengths (no I/O)	—
`DAP_JTAG_IDCODE`	SIO bit-bang (`JTAG_ReadIDCode`)	—
`DAP_Transfer` , `DAP_TransferBlock`	SIO bit-bang (`JTAG_IR` , `JTAG_Transfer`)	SIO bit-bang (`SWD_Transfer`)
`DAP_WriteABORT`	SIO bit-bang	SIO bit-bang
`DAP_SWJ_Sequence`	SIO bit-bang	SIO bit-bang
`DAP_SWJ_Pins`	SIO (see § 13.1)	SIO
`DAP_SWD_Sequence`	—	SIO bit-bang
Vendor `0x80`	Packet timing counters	same

In JTAG mode OpenOCD drives the target almost entirely with `DAP_JTAG_Sequence` (it builds its own IR/DR scans from TMS/TDI sequences), which is why accelerating that one command gives most of the benefit.

7.3 Two clocks from one `adapter speed`

`DAP_SWJ_Clock(clock_hz)` updates both engines:

- **Bit-bang (SIO paths):** the ARM formula `delay = ceil((CPU_CLOCK/2) / clock) − IO_PORT_WRITE_CYCLES` , divided by `DELAY_SLOW_CYCLES` (3 cycles per delay-loop iteration), stored in `DAP_Data.clock_delay` . The "fast" path (no delay) is only chosen at or above `MAX_SWJ_CLOCK(0)` = 62.5 MHz, i.e. practically never. The real bit-bang rate is below the requested one because the loop overhead around each pin access is not part of the formula.
- **PIO:** `pio_jtag_set_clock()` → `div = clk_sys / (4 × clock)` , clamped to 1 … 65535, applied with `pio_sm_set_clkdiv()` . The value is remembered in `g_pio_jtag.clock_hz` , so a later `DAP_Connect(JTAG)` restarts the PIO at the last requested speed even if `adapter speed` was set before the connect.

8. The PIO JTAG engine

8.1 Resources

Item	Value
PIO block	First free state machine of `pio0`, else `pio1` (`pio_claim_unused_sm`)
Program	`jtag_tdo`, 13 instructions, loaded with `pio_add_program()`
Side-set	1 bit, mandatory, → TCK (GP2)
`SET` pins	1 → TMS (GP3)
`OUT` pins	1 → TDI (GP6)
`IN` pins	base TDO (GP7), input synchroniser bypassed (`input_sync_bypass`)
OSR	shift right (LSB first), **autopull off**, threshold 8
ISR	shift right (LSB first), **autopush on**, threshold 8
FIFOs	4-word TX, 4-word RX (not joined)
Clock divider	`clk_sys / (4 × f_TCK)`

State is kept in the global `g_pio_jtag` (`pio_jtag_t`: PIO, SM, program offset, `enabled`, `pins_pio`, `clock_hz`, pin numbers).

8.2 The program (`src/jtag.pio`)

```
.program jtag_tdo
.side_set 1                         ; side-set = TCK
.wrap_target
    pull block          side 1      ; header word, TCK held high
    out x, 7            side 1      ; X = TCK count - 1 (0..63)
    out y, 1            side 1      ; Y = TMS level
    jmp !y tms_low      side 1
    set pins, 1         side 1      ; TMS = 1 while TCK is high
    jmp bits            side 1
tms_low:
    set pins, 0         side 1      ; TMS = 0 while TCK is high
bits:
    out null, 32        side 1      ; discard rest of OSR -> next pull ifempty fetches data
loop:
    pull ifempty block  side 0      ; TCK low; new TDI byte every 8 bits
    out pins, 1         side 0      ; TDI = next bit
    in pins, 1          side 1      ; TCK high; sample TDO
    jmp x--, loop       side 1
    push block          side 1      ; last (partial or empty) TDO word
.wrap
```

8.3 Stream protocol

One `DAP_JTAG_Sequence` *entry* (info byte + TDI bytes) becomes:

TX FIFO (CPU → PIO)

Word	Content
Header	bits 0–6: TCK count − 1 (0…63), bit 7: TMS level
Data × ⌈n/8⌉	one TDI byte per 32-bit word, LSB first

RX FIFO (PIO → CPU): ⌊n/8⌋ + 1 words: every full TDO byte (autopush at 8 bits; the byte sits in bits 31…24 because the ISR shifts right), then one word from the final `push block` holding the last r = n mod 8 bits in bits 31…32−r. When r = 0 that last word is empty (autopush has just emptied the ISR) but is still pushed, which keeps the word count per entry fixed and lets the CPU stay in step.

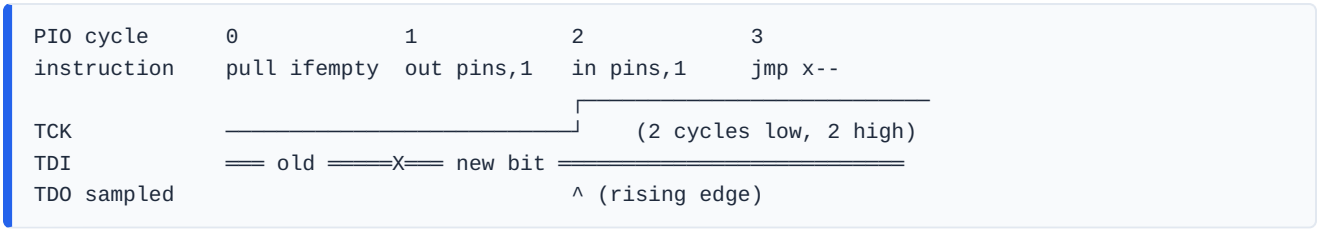
The CMSIS-DAP info byte is translated by `entry_header()`: TCK count = `info & 0x3F` (0 means 64), TMS = `info & 0x40`. The TDO-capture flag (`info & 0x80`) never reaches the PIO: the PIO always samples TDO, and the CPU decides whether to keep the bytes.

Design notes:

- **Why autopull is off.** With autopull, the 24 unused bits of the header word would remain in the OSR and be shifted out as TDI, and the next entry's header could be prefetched as data. `out null, 32` empties the OSR so that the first `pull ifempty` of the bit loop fetches a fresh data word, and so does every eighth bit thereafter.
- **Why `push block`.** A non-blocking push would drop a word if the CPU fell behind; blocking stalls the state machine instead (with TCK high, a legal JTAG clock stretch).
- **TMS from the stream.** TMS is set by the PIO from the header while TCK is high, before the entry's first falling edge, so successive entries with different TMS run back to back without the CPU touching a pin. This was the change that turned the PIO from slower than bit-banging into twice as fast (§ 11).

8.4 Bit timing

Each bit takes 4 PIO cycles; side-set changes TCK at the start of an instruction, even when that instruction stalls.



- **f_TCK = clk_sys / (4 · div)**, 50 % duty cycle (2 cycles low, 2 high).
- **TDI** changes about one PIO cycle after the falling edge, so it is set up roughly one PIO cycle (¼ TCK period) before the rising edge and held through it.
- **TDO** is sampled at the rising edge. The target updates TDO on the falling edge, so the data has had ½ TCK period to settle; bypassing the input synchroniser removes the 2-cycle sampling delay that would otherwise eat into that margin at high clocks.
- **Per-entry overhead:** the header path costs 6 (TMS = 0) or 7 (TMS = 1) cycles plus 1 for `push`, all with TCK high: an n-bit entry takes 4n + 7…8 PIO cycles, i.e. about two extra TCK periods of "TCK high".
- **Starvation:** if the TX FIFO is empty, `pull ifempty block` stalls with TCK **low**; if the RX FIFO is full, `push block` stalls with TCK high. In both cases the clock is only stretched: JTAG is fully static, so this is harmless, and it is what happens when core 1 cannot keep up.

Divider examples (clk_sys = 125 MHz, the value `CPU_CLOCK` assumes):

`adapter speed` (kHz)	div	Actual TCK
1000	31.25	1 MHz average (fractional divider)
4000	7.8125	4 MHz average (fractional)
10000	3.125	10 MHz average (fractional)
15000	2.083	15 MHz average (fractional)
15625	2	15.625 MHz exact
31250	1	31.25 MHz — the maximum (div is clamped to ≥ 1)

The RP2040 divider is 16.8 fixed point: a non-integer value produces the requested *average* frequency, with individual PIO cycles (and so TCK half-periods) differing by one system clock.

8.5 The CPU side: `pio_jtag_sequences()`

`DAP_JTAG_Sequence()` (`src/DAP.c`) passes the whole command (entry count, request pointer, response pointer) to `pio_jtag_sequences()` when the PIO is enabled. The function walks the request with **two independent cursors** in a single loop:

```
while entries or bytes remain on either side:
    TX cursor:  if TX FIFO not full:
                    send the next TDI byte of the current entry, or
                    the next entry's header (and compute its byte count)
    RX cursor:  if no RX words outstanding: parse the next entry's info byte
                    -> words = n/8 + 1, rem = n%8, capture = info & TDO
                if RX FIFO not empty:
                    pop a word; if capture:
                        full-byte word -> *out++ = word >> 24
                        last word      -> *out++ = word >> (32 - rem)  (if rem != 0)
return (request bytes consumed << 16) | response bytes written
```

The TX cursor runs ahead by up to the FIFO depth, so the PIO always has the next header or data byte queued; the CPU never waits for an entry to finish. The single-entry wrapper `pio_jtag_sequence()` (used by `JTAG_Sequence()` in `JTAG_DP.c`) packs one entry into a small buffer and calls the same routine.

8.6 Sharing the pins with the bit-bang paths

Only `DAP_JTAG_Sequence` uses the PIO. The other JTAG commands still run the ARM bit-bang code, which writes the pins through SIO. Instead of adding a check to every per-bit pin macro (which SWD would pay for on every bit), the hand-over happens once per command:

- **PIO → SIO:** `PIN_TCK_TDI_TO_SIO()` (`DAP_config.h`) is called at the entry of `JTAG_IR()`, `JTAG_Transfer()`, `JTAG_ReadIDCode()`, `JTAG_WriteAbort()`, `DAP_SWJ_Sequence()` and, when it selects TCK/TMS/TDI, `DAP_SWJ_Pins()`. It calls `pio_jtag_pins_to_sio()`, which reads the current levels with `gpio_get_all()`, loads them into the SIO output latch, sets the SIO direction to output and only then switches the function mux to SIO. Between commands the PIO is stalled on `pull block` with TCK high, so the levels are well defined and nothing glitches.
- **SIO → PIO:** `pio_jtag_sequences()` begins with `pio_jtag_pins_to_pio()`, which switches the mux back (`pio_gpio_init`). The PIO's side-set still holds TCK high, as the bit-bang code leaves it; TMS and TDI are rewritten by the next header and data bit before they matter.
- `g_pio_jtag.pins_pio` records the current owner, so both directions are no-ops when nothing changes.

nTRST and nRESET always stay on SIO.

8.7 Lifecycle

Event	Function	Effect
`DAP_Connect(JTAG)`	`pio_jtag_init(g_pio_jtag.clock_hz)`	Claims an SM, loads the program, configures pins/shifts/divider, enables the SM. If already enabled, only updates the divider
`DAP_SWJ_Clock`	`pio_jtag_set_clock()`	New divider (stored even when the PIO is off)
`DAP_Connect(SWD)` / `DAP_Disconnect`	`pio_jtag_deinit()`	Disables the SM, removes the program, unclaims, returns TCK/TDI/TDO/TMS to SIO high-Z

If no state machine or no instruction memory is free, `pio_jtag_init()` returns false, `pio_jtag_is_enabled()` stays false, and `DAP_JTAG_Sequence` falls back to the original per-entry bit-bang loop.

9. The SWD path

SWD does not use the PIO at all: `PORT_SWD_SETUP()` releases it and drives SWCLK/SWDIO through SIO. `SW_DP.c` is the ARM reference code (request, turnaround, ACK, 32 data bits + parity) using the single-cycle SIO pin macros; SWDIO direction changes with `gpio_set_dir_in_masked` / `gpio_set_dir_out_masked`. The README records a verified `smp-test1` run on a Raspberry Pi Pico target over SWD at 15 MHz.

10. The USB CDC-ACM UART bridge (`src/cdc_uart.c`)

- `uart1` on GP4 (TX) / GP5 (RX), pull-ups on both, 115200 8N1 by default.
- The `UART` thread (core 0, high priority) polls the PL011 FIFO and the CDC buffers. Its period comes from the baud rate: the time to fill half the 32-byte FIFO, `interval = max(1 ms, 16 × 10 bits / baud)`.
- **CDC callbacks never touch the UART.** `tud_cdc_line_coding_cb`, `tud_cdc_line_state_cb` and `tud_cdc_send_break_cb` run inside `tud_task()` (in `TUD`, with `usb_mutex` held). They store a request (new line coding, break) and post `uart_sem`; the `UART` thread applies it. This keeps UART reconfiguration out of the USB thread and out of the mutex.
- Line coding: 5–8 data bits; parity none/odd/even; 1 or 2 stop bits (1.5 is mapped to 2, the PL011 has no 1.5).
- Break: `0` ends it, `0xFFFF` holds it, other values time it in ms.
- When the host deasserts both DTR and RTS, polling stops and the thread sleeps on `uart_sem` until the port is opened again.
- The UART FIFO is drained even with no host connected, so stale target output does not accumulate.

10.1 Status LED (`src/led.c`, `src/led.h`)

The Pico's on-board LED (GP25, `PROBE_USB_CONNECTED_LED`) is **on all the time** and **blinks rapidly (10 Hz: 50 ms on, 50 ms off) while TCK/SWCLK is in use**.

- **Marking activity (core 1).** `DAP_ProcessCommand()` calls `led_tck_activity()` before every command that clocks TCK/SWCLK: `DAP_SWJ_Sequence`, `DAP_SWD_Sequence`, `DAP_JTAG_Sequence`, `DAP_JTAG_IDCODE`, `DAP_Transfer`, `DAP_TransferBlock`, `DAP_WriteABORT` (also when they come inside `DAP_ExecuteCommands`). It only stores `time_us_32()` in `led_tck_last_us`: one store, no lock, no measurable cost.

- **Driving the LED (core 0).** `led_init()` (called from `os_main()` after `board_init()`) makes GP25 an output, turns it on and starts an SDK repeating timer (`add_repeating_timer_ms`, 50 ms) on the default alarm pool. Its callback runs in the timer IRQ: if a clocking command ran in the last 100 ms (`LED_TCK_HOLD_US`) it toggles the LED, otherwise it sets it on. The hold time makes even a single short command visible as a blink.
- **Why a timer IRQ and not a thread.** During a transfer the `TUD` thread spins on core 0 above normal priority, and the `DAP` thread sleeps when the host is idle; a thread would leave the LED frozen in one state in either case. The callback uses no kernel API, only SIO GPIO writes.
- An OpenOCD session that polls the target keeps clocking TCK, so the LED keeps blinking while OpenOCD is connected and polling; it goes back to steady on once the host stops issuing clocking commands.
- **Verified on hardware:** steady on when idle, blinking while the host clocks TCK/SWCLK.

11. Performance summary

Figures measured by the project on a Raspberry Pi 4 B target over JTAG with OpenOCD, 512-byte packets, smp_mem_test with `SEMI_PRINTS=1` (from `README.md`, § "PIO results"):

Probe firmware	`adapter speed`	`load_image` (74,888 B)	JTAG work / packet	Load cycle / packet	Whole test
JTAGprobe_smp, CPU bit-banging	4000	1,528 ms (49.0 KB/s)	581 µs	1,745 µs	308.1 s
PIO, TMS via SIO, wait per entry	10000	1,578 ms	621 µs	1,794 µs	315.3 s
PIO, TMS in stream (this firmware)	**10000**	**1,286 ms (58.2 KB/s)**	**292 µs**	**1,450 µs**	**291.5 s**
PIO, TMS in stream	15000	1,287 ms	291 µs	1,448 µs	285.1 s

What the numbers show:

- Streaming entries with TMS in the stream **halves the JTAG work per packet** (581 → 292 µs). A PIO that waits after each entry is *slower* than bit-banging, because OpenOCD sends hundreds of short entries per packet.
- 15 MHz is no faster than 10 MHz: TCK is no longer the bottleneck; most likely core 1 feeding the FIFOs one byte per word is (not measured separately).
- The rest of a ~1,450 µs load cycle (~920 µs) is receiving the request in 64-byte USB packets plus the host's turnaround — the largest remaining cost, outside the probe's CPUs.
- Historical context (README): stock debugprobe 2.0.0 ≈ 1.05 KB/s; this firmware ≈ 58 KB/s on the same kind of load.

12. Using the probe

12.1 Flashing

Hold BOOTSEL, plug in, copy `build/JTAGprobe_smp_pio_std.uf2` to the `RPI-RP2` drive.

12.2 OpenOCD (JTAG)

```
adapter driver cmsis-dap
cmsis-dap backend usb_bulk          ;# CMSIS-DAP v2
cmsis-dap vid_pid 0x2e8a 0x000c
transport select jtag
adapter speed 10000                 ;# kHz; -> DAP_SWJ_Clock -> PIO divider
source [find target/<your-target>.cfg]
```

For SWD use `transport select swd` (bit-banged, PIO released).

12.3 UART console

```
picocom -b 115200 /dev/ttyACM0      # baud/format follow the host's settings
```

12.4 Probe-side timing

```
cmsis-dap cmd 0x80 0xff             ;# reset counters
# ... run a load ...
cmsis-dap cmd 0x80 0                ;# packets
cmsis-dap cmd 0x80 2                ;# exec_us
```

13. Observations and known caveats

These come from reading the current source. Not all of them are defects, but users and maintainers should know about them.

13.1 `DAP_SWJ_Pins` bit layout differs from the CMSIS-DAP specification

The CMSIS-DAP protocol defines the `DAP_SWJ_Pins` select/value/response bits as TCK = 0, TMS = 1, TDI = 2, TDO = 3, nTRST = 5, nRESET = 7. In this firmware the `DAP_SWJ_*` macros in `include/DAP.h` hold the **GPIO numbers** (TCK 2, TMS 3, TDI 6, TDO 7, nTRST 0, nRESET 1), and `DAP_SWJ_Pins()` uses them as bit positions. Consequences:

Host sets bit	Spec meaning	What this firmware does
0	TCK	nTRST (`PIN_nTRST_OUT`, a no-op, see § 13.2)
1	TMS	**nRESET**
2	TDI	**TCK**
3	TDO (read-only)	**TMS**
5	nTRST	nothing
7	nRESET	nothing

So an OpenOCD hardware reset through `DAP_SWJ_Pins` (SRST, bit 7) does not toggle nRESET, and the response byte is not in spec order. The git history shows the spec positions were tried (`01685db`) and reverted (`5c31a12`) because OpenOCD then read DPIDR `0x00000000` over SWD on an RP2040 target; the commit does not identify the cause of that failure. The proper fix is to use spec bit positions in `DAP_SWJ_Pins()` while keeping the GPIO numbers in `PROBE_PIN_*`, then find out why SWD failed with that change. Until then, prefer `reset_config none` or a target-side software reset (e.g. `AIRCR.SYSRESETREQ`) with OpenOCD.

13.2 nTRST is not host-controllable

`PIN_nTRST_OUT()` is an empty stub and `PIN_nTRST_IN()` returns 0. GP0 is driven high by `pio_jtag_init()` when JTAG connects and floats (high-Z) in SWD mode, but no DAP command can pulse it. Use a TAP reset through TMS (five TCK cycles with TMS high), which OpenOCD does by default.

13.3 Bit-bang timing assumes a 125 MHz system clock

The PIO divider is computed from the actual `clock_get_hz(clk_sys)`, but the bit-bang delay uses the constant `CPU_CLOCK = 125 MHz`. If the SDK or board configuration runs the RP2040 at another frequency, SWD and the non-sequence JTAG commands run proportionally faster or slower than requested, while `DAP_JTAG_Sequence` stays correct.

13.4 Mixed clock rates within one JTAG session

At a given `adapter speed`, `DAP_JTAG_Sequence` runs at the exact PIO rate while `DAP_Transfer` / `DAP_TransferBlock` in JTAG mode run at the (slower) bit-bang rate. Both are correct JTAG; only throughput differs. Moving `JTAG_Transfer` to the PIO stream is the natural next acceleration step for hosts (pyOCD, Keil) that use `DAP_Transfer` in JTAG mode.

13.5 Leftover code paths

- `os_main()` contains a non-threaded loop (`THREADED` = 0) that polls IDCODE and sends HID reports; with `THREADED` = 1 it is dead code.
- `tud_hid_set_report_cb()` / `tud_hid_get_report_cb()` belong to the CMSIS-DAP v1 (HID) build only (`PROBE_DEBUG_PROTOCOL == PROTO_DAP_V1`).
- `probe_info` / `probe_debug` / `probe_dump` are compiled out (`#if false`).

14. Source map

File	Content
`CMakeLists.txt`	Pico SDK project, µOS++ port inclusion, PIO header generation, defines, `copy_to_ram`
`include/DAP_config.h`	CMSIS-DAP configuration, pin numbers, SIO pin primitives, `PORT_*_SETUP`, `PIN_TCK_TDI_TO_SIO`
`include/DAP.h`	CMSIS-DAP IDs and constants, `DAP_SWJ_*`, delay primitives
`include/pio_jtag.h`, `src/pio_jtag.c`	PIO JTAG API, init/deinit, clock, pin hand-over, `pio_jtag_sequences()`
`src/jtag.pio`	The `jtag_tdo` PIO program
`include/board_pico_config.h`	LED, product string, UART pins/instance/baud
`src/main.c`	`os_main()`, thread creation and affinity, `TUD` thread, tinyusb hooks, MS OS 2.0 control request
`src/tusb_edpt_handler.c/.h`	CMSIS-DAP USB class driver, request reassembly (`dap_req_len`), rings, DAP thread, timing counters
`src/usb_descriptors.c`	Device/config/string/MS OS 2.0 descriptors
`src/tusb_config.h`	tinyusb configuration
`src/cdc_uart.c/.h`	UART bridge thread and CDC callbacks
`src/led.c/.h`	Status LED: on, 10 Hz blink while TCK/SWCLK is in use
`src/DAP.c`	CMSIS-DAP command processing (PIO hook in `DAP_JTAG_Sequence`, divider update in `DAP_SWJ_Clock`)
`src/JTAG_DP.c`	JTAG bit-bang primitives (`JTAG_IR`, `JTAG_Transfer`, IDCODE, ABORT), PIO hook in `JTAG_Sequence`
`src/SW_DP.c`	SWD bit-bang primitives
`src/SWO.c`	SWO (compiled, disabled by configuration)
`src/DAP_vendor.c`	Vendor command `0x80` (packet timing)
`src/get_serial.c/.h`	USB serial number from the flash unique ID
`src/probe.h`, `src/probe_os.h`	Protocol selection, debug-print macros, shared kernel objects
`src/cmsis-plus/os-app-config.h`	µOS++ configuration
`micro-os-plus/micro-os-plus-iii/`	µOS++ III SMP kernel (unmodified, `smp` @ `931f1380`)
`micro-os-plus/micro-os-plus-iii-rp2040/`	RP2040 SMP port: `os-core-rp2040.cpp`, port headers, CMake
`CMSIS_6/`	ARM CMSIS 6 (submodule), Core headers
`README.md`, `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md`	User guide, measurements and optimisation history